人間のための RuboCop プラグインが AI のハーネスになるまで

Gotanda.rb#70 / Takeshi KOMIYA / 2026-10-28

TIMEINTERMEDIA

自己紹介



- 小宮 健（Takeshi KOMIYA）
  - @tk0miya
- CTO of TIME INTERMEDIA, Inc.
- OSS Developer
  - Sphinx（Python 製ドキュメンテーションツール）
  - rbs_rails メンテナ（Rails アプリ用型生成ツール）
- 書籍: Sphinx をはじめよう（オライリー・ジャパン）

作ってきた RuboCop プラグイン

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- rubocop-herb

rubocop-rbs_inline

rbs-inline の型注釈の書き方を揃える

```
# bad: 同じ型を二重に書いている
# @rbs name: String
#: (String) -> String
def greet(name) = "Hello, #{name}"
```

rubocop-rbs_inline

rbs-inline の型注釈の書き方を揃える

```
# bad: 同じ型を二重に書いている
# @rbs name: String
#: (String) -> String
def greet(name) = "Hello, #{name}"

# good
# @rbs name: String
# @rbs return: String
def greet(name) = "Hello, #{name}"
```

rubocop-rspec-unused-let

使われていない let を見つける

```ruby
RSpec.describe Foo do
  let(:used)   { 1 }
  let(:unused) { 2 } # 誰も参照していない

  it { expect(used).to eq(1) }
end
```

rubocop-rspec-unused-let

使われていない let を見つける

```ruby
RSpec.describe Foo do
  let(:used)   { 1 }
  let(:unused) { 2 } # 誰も参照していない

  it { expect(used).to eq(1) }
end
```

rubocop-numbered-params

1 行ブロックは numbered parameter で

```
# bad
users.map { |user| user.name }
```

rubocop-numbered-params

1 行ブロックは numbered parameter で

```
# bad
users.map { |user| user.name }

# good
users.map { _1.name }
```

rubocop-prefer_it_parameter

1 行ブロックは it で

```
# bad
users.map { |user| user.name.upcase }
```

rubocop-prefer_it_parameter

1 行ブロックは it で

```
# bad
users.map { |user| user.name.upcase }

# good
users.map { it.name.upcase }
```

rubocop-rspec-structure

context と example を分離する（Jev の試作）

```
# bad
it "when the user is an admin, allows deletion" do
end
```

rubocop-rspec-structure

context と example を分離する（Jev の試作）

```
# bad
it "when the user is an admin, allows deletion" do
end

# good
context "when the user is an admin" do
  it "allows deletion" do
  end
end
```

rubocop-instance_variable_access

インスタンス変数は reader 経由で読む

```
# bad
def full_name = "#{@first_name} #{@last_name}"
```

rubocop-instance_variable_access

インスタンス変数は reader 経由で読む

```ruby
# bad
def full_name = "#{@first_name} #{@last_name}"

# good
attr_reader :first_name, :last_name

def full_name = "#{first_name} #{last_name}"
```

RuboCop プラグインを作った目的

- AI は**新しい記法**を知らない・使わない
  - `_1`（numbered-params）
  - `it`（prefer_it_parameter）
  - 型注釈（rbs_inline）
- コーディングルールを**自己検証**可能に
  - context 構造の強制（rspec-structure）
  - reader 利用の強制（instance_variable_access）
- リファクタリング時の**ゴミ**を検知
  - 未使用 let 変数の検知（rspec-unused-let）

RuboCop プラグインの役割が変わってきた

- 当初: 人間と AI の両方が書く前提
  - 均質なコード、うっかりミスを減らす
- 実装が AI 任せに: 良いコードを出させる
- 最近: **AI をサポートする RuboCop プラグイン**
  - ゴミの検出、レビューしやすい構造

RuboCop プラグインの役割が変わってきた

- 当初: 人間と AI の両方が書く前提
  - 均質なコード、うっかりミスを減らす
- 実装が AI 任せに: 良いコードを出させる
- 最近: **AI をサポートする RuboCop プラグイン**
  - ゴミの検出、レビューしやすい構造
  - 自己検証可能な規約を提供する（= ハーネス）
  - プロンプト < RuboCop プラグイン

Pencils Down



https://x.com/dhh/status/2102936073642869121

Pencils Down



https://x.com/nysalor/status/2111091190103966017

コード品質はどれくらい重要になる？

- 答えは不明だが、**整え続けている**
  - いまでもコードは書く回数よりも、読む回数が多い（はず）
  - 整ったコード、コンパクトなコードには価値がある（はず）
  - 新しい記法に効果があるかは自信なし

実はまだ紹介していないプラグインがあります

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- rubocop-herb

実はまだ紹介していないプラグインがあります

- rubocop-rbs_inline
- rubocop-rspec-unused-let
- rubocop-numbered-params
- rubocop-prefer_it_parameter
- rubocop-rspec-structure
- rubocop-instance_variable_access
- **rubocop-herb**

宣伝

仙台Ruby会議 #03 で rubocop-herb の話をします



宣伝

仙台Ruby会議 #03 で rubocop-herb の話をします



まとめ

- 規約はプロンプトより **RuboCop プラグイン**にする
- 人間のための RuboCop プラグインは AI にも効く
- 最初から AI のために作るのもアリ

ご清聴ありがとう
ございました

